

Guía Definitiva de Diseño: Diagrama de Clases UML para "ASCEND"

Propósito del Documento:

Esta guía proporciona la especificación técnica completa para construir el **Diagrama de Clases UML** del sistema **SGDM ASCEND**. Incluye el desglose de todas las clases del backend en PHP (Modelos, Controladores, Motores de Torneo y Servicios de Infraestructura), con la definición exacta de sus **visibilidades, atributos tipados, métodos con parámetros y retornos, y relaciones de asociación, herencia, composición y dependencia**.

1. Simbología y Normas Estándar del UML de Clases

1.1 Visibilidad de Atributos y Métodos

Símbolo	Significado	Ámbito de Acceso	Ejemplo en PHP
+	Público (public)	Accisible desde cualquier clase o script.	<code>+login(): void</code>
-	Privado (private)	Accisible solo dentro de la propia clase.	<code>-contrasenaHash: string</code>
#	Protegido (protected)	Accesible por la clase y sus subclases derivadas.	<code>#id: int</code>

1.2 Tipos de Relaciones en UML

Relación UML	Conector Visual	Significado en SGDM ASCEND	Ejemplo en el Sistema
Generalización / Herencia		Una clase hija extiende de una clase padre.	<code>PerfilJugador</code> extiende de <code>Usuario</code> <code>AuthControlador</code> extiende de <code>ControladorBase</code>
Composición		Relación fuerte de vida: la parte no existe sin el todo.	<code>Torneo</code> contiene <code>Encuentro</code> (si borro el torneo, mueren sus encuentros).
Agregación		Relación débil: la parte puede existir independientemente.	<code>Rol</code> se asigna a <code>Usuario</code> (el rol existe aunque se borre un usuario).
Dependencia / Uso		Una clase utiliza a otra temporalmente en un método.	<code>AuthControlador</code> usa <code>Validador</code> y <code>Sesion</code> .

2. Inventario Detallado de Clases por Capas Arquitectónicas

Capa A: Modelos de Entidad ([codigo_fuente/modelos/](#))

1. Usuario (Superclase de Identidad)

- **Estereotipo:** <<Entity Model>>
- **Atributos:**
 - #id: int
 - #email: string
 - #contrasenaHash: string
 - #nombreCompleto: string
 - #telefono: string
 - #fotoPerfilUrl: string
 - #rolId: int
 - #estaActivo: bool
 - #fechaRegistro: DateTime
- **Métodos:**
 - +obtenerPorId(id: int): Usuario
 - +buscarPorEmail(email: string): array
 - +crear(datos: array): int
 - +actualizarUsuario(id: int, datos: array): bool
 - +cambiarEstadoUsuario(id: int, estado: int): bool
 - +eliminarUsuario(id: int): bool
 - +obtenerTodosUsuariosConRoles(): array

2. PerfilJugador (Subclase Subtipo 1:1)

- **Estereotipo:** <<Subtype Entity>>
- **Atributos:**
 - -usuarioId: int
 - -apodoGamertag: string
 - -biografia: string
 - -nivel: int
 - -puntosExperiencia: int
 - -pais: string
 - -discordTag: string
- **Métodos:**
 - +obtenerPerfil(usuarioId: int): array
 - +actualizarGamertag(usuarioId: int, tag: string): bool
 - +sumarExperiencia(usuarioId: int, xp: int): bool
- **Relación:** Generalización (--|>) con Usuario.

3. PerfilOrganizador (Subclase Subtipo 1:1)

- **Estereotipo:** <<Subtype Entity>>
- **Atributos:**
 - -usuarioId: int
 - -nombreOrganizacion: string
 - -bioOrganizacion: string
 - -sitioWeb: string

- -telefonoContacto: string
- -verificadoOficial: bool
- **Métodos:**
 - +obtenerOrganizador(usuarioId: int): array
 - +solicitarVerificacion(usuarioId: int): bool
- **Relación:** Generalización (--|>) con *Usuario*.

4. **Equipo** (*Escuadra Deportiva*)

- **Estereotipo:** <<Entity Model>>
- **Atributos:**
 - #id: int
 - #nombreEquipo: string
 - #escudoUrl: string
 - #creadoPor: int
 - #codigoInvitacion: string
 - #activo: bool
- **Métodos:**
 - +crearEquipo(datos: array): int
 - +obtenerPorId(id: int): Equipo
 - +transferirCapitania(equipoId: int, nuevoCapitanId: int): bool
 - +contarTotalEquipos(): int

5. **EquipoMiembro** (*Entidad de Asociación N:M*)

- **Estereotipo:** <<Association Entity>>
- **Atributos:**
 - #equipoId: int
 - #usuarioId: int
 - #fechaUnion: DateTime
 - #numeroCamiseta: int
 - #posicion: string
 - #esActivo: bool
- **Métodos:**
 - +agregarMiembro(equipoId: int, usuarioId: int): bool
 - +removeMiembro(equipoId: int, usuarioId: int): bool
 - +obtenerMiembrosPorEquipo(equipoId: int): array

6. **Torneo** (*Aggregate Root / Núcleo del Sistema*)

- **Estereotipo:** <<Aggregate Root Entity>>
- **Atributos:**
 - #id: int
 - #nombre: string
 - #juegoId: int
 - #organizadorId: int
 - #modalidadId: int

- #sistemaPuntuacionId: int
- #formato: string
- #estado: string
- #cupoMaxEquipos: int
- #fechaInicio: Date
- #fechaFin: Date

- **Métodos:**

- +crearTorneo(datos: array): int
- +cambiarEstado(id: int, nuevoEstado: string): bool
- +contarTorneosActivos(): int
- +obtenerUltimosTorneosCreados(limite: int): array

7. ParticipanteTorneo (*Inscripción Polimórfica*)

- **Estereotipo:** <<Association Entity>>

- **Atributos:**

- #id: int
- #torneoId: int
- #tipo: string
- #referenciaId: int
- #nombre: string
- #estado: string

- **Métodos:**

- +inscribir(torneoId: int, tipo: string, refId: int): bool
- +confirmarCupo(id: int, confirmadoPor: int): bool

8. Encuentro (*Partido / Fixture*)

- **Estereotipo:** <<Entity Model>>

- **Atributos:**

- #id: int
- #torneoId: int
- #ronda: int
- #participanteLocalId: int
- #participanteVisitanteId: int
- #resultadoLocal: float
- #resultadoVisitante: float
- #estado: string
- #participanteGanadorId: int

- **Métodos:**

- +registrarMarcador(id: int, local: float, visita: float): bool
- +validarResultado(id: int, arbitroId: int): bool

9. PosicionTorneo (*Tabla de Clasificación*)

- **Estereotipo:** <<Entity Model>>

- **Atributos:**

- #torneoId: int
- #participanteId: int
- #partidosJugados: int
- #partidosGanados: int
- #partidosEmpatados: int
- #partidosPerdidos: int
- #puntosFavor: float
- #puntosContra: float
- #diferenciaGoles: float
- #puntos: int
- **Métodos:**
 - +obtenerTablaOrdenada(torneoId: int): array

10. Juego, Rol, Permiso, PoliticaContrasena, Auditoria

- **Juego:** #id: int, #nombre: string, #categoria: string → +obtenerTodos(): array, +crearJuego(datos: array): bool, +actualizarJuego(id: int, datos: array): bool, +cambiarEstado(id: int): bool
 - **PoliticaContrasena:** #id: int, #longitudMinima: int, #expiracionDias: int → +obtenerPoliticaVigente(): array, +guardarPolitica(datos: array, usuarioId: int): bool, +validarPassword(password: string): array
 - **Rol:** #id: int, #nombreRol: string, #nivelPermiso: int → +obtenerPermisos(rolId: int): array
 - **Auditoria:** -id: int, -usuarioId: int, -accion: string, -descripcion: string → +registrar(accion: string, descripcion: string): void, +obtenerTodos(): array
-

Capa B: Motores Algorítmicos de Torneos (*Tournament Engines*)

1. ModuloLiga (*Algoritmo Round-Robin / Berger*)

- **Estereotipo:** <<Tournament Engine>>
- **Atributos:**
 - -puntosVictoria: float = 3.0
 - -puntosEmpate: float = 1.0
 - -puntosDerrota: float = 0.0
- **Métodos:**
 - +generarFixtureTodosContraTodos(torneoId: int): bool
 - +recalcularTablaPosiciones(torneoId: int): bool

2. ModuloEliminacion (*Algoritmo de Llaves / Árbol Binario*)

- **Estereotipo:** <<Tournament Engine>>
- **Atributos:**
 - -totalRondas: int
 - -tieneTercerPuesto: bool
- **Métodos:**
 - +generarArbolLlaves(torneoId: int): bool

- `+avanzarGanador(encuentroId: int, ganadorId: int): bool`

3. ModuloSuizo (*Algoritmo Swiss-System / Buchholz*)

- **Estereotipo:** `<<Tournament Engine>>`
 - **Atributos:**
 - `-totalRondas: int`
 - `-criterioDesempate: string = "Buchholz"`
 - **Métodos:**
 - `+generarRondaSuiza(torneoId: int, rondaNumero: int): bool`
 - `+calcularBuchholz(torneoId: int, participanteId: int): float`
-

Capa C: Controladores MVC (`codigo_fuente/controladores/`)

1. AdminControlador

- **Estereotipo:** `<<Controller>>`
- **Métodos:**
 - `+dashboard(): void`
 - `+obtenerUltimosJugadores(limite: int): array`
 - `+obtenerUltimosOrganizadores(limite: int): array`

2. UsuarioControlador

- **Estereotipo:** `<<Controller>>`
- **Métodos:**
 - `+index(): void`
 - `+crear(): void`
 - `+guardar(): void`
 - `+editar(id: int): void`
 - `+actualizarUsuario(): void`
 - `+cambiarEstado(): void`
 - `+eliminar(): void`
 - `+verComo(): void`
 - `+volverAdmin(): void`

3. JuegoControlador

- **Estereotipo:** `<<Controller>>`
- **Métodos:**
 - `+index(): void`
 - `+crear(): void`
 - `+editar(): void`
 - `+actualizar(): void`
 - `+cambiarEstado(): void`

4. PoliticaContraseñaControlador

- **Estereotipo:** <<Controller>>
- **Métodos:**
 - +index(): void
 - +guardar(): void

5. AuditoriaControlador

- **Estereotipo:** <<Controller>>
- **Métodos:**
 - +index(): void

Capa D: Servicios e Infraestructura

1. Conexion (Patrón Singleton PDO)

- **Estereotipo:** <<Singleton Pattern>>
- **Atributos:**
 - -instancia: Conexion = null
 - -pdo: PDO
- **Métodos:**
 - +getInstance(): Conexion
 - +getBD(): PDO

2. Sesion (Servicio de Seguridad RBAC)

- **Estereotipo:** <<Security Service>>
- **Métodos:**
 - +iniciarLogin(usuario: array): void
 - +validarAdmin(): void
 - +esAdmin(): bool
 - +destruirSesion(): void

3. Validador (Servicio de Sanitización)

- **Estereotipo:** <<Validation Service>>
- **Métodos:**
 - +requerido(valor: string): bool
 - +emailValido(email: string): bool
 - +enteroEnRango(v: int, min: int, max: int): bool
 - +sanitizar(cadena: string): string

3. Diagrama Mermaid Oficial de Clases UML

```
classDiagram
    %% =====
    %% HERENCIA Y SUBTIPOS DE USUARIO
```

```

%% =====
class Usuario {
    <>
    #int id
    #string email
    #string contrasenaHash
    #string nombreCompleto
    #string telefono
    #int rolId
    #bool estaActivo
    #DateTime fechaRegistro
    +obtenerPorId(id) Usuario
    +obtenerPorEmail(email) Usuario
    +crear(datos) int
    +actualizar(id, datos) bool
    +cambiarEstado(id, activo) bool
    +verificarPassword(passwordPlana) bool
}

class PerfilJugador {
    <>
    -int usuarioId
    -string apodoGamertag
    -string biografia
    -int nivel
    -int puntosExperiencia
    -string pais
    +obtenerPerfil(usuarioId) array
    +actualizarGamertag(usuarioId, tag) bool
    +sumarExperiencia(usuarioId, xp) bool
}

class PerfilOrganizador {
    <>
    -int usuarioId
    -string nombreOrganizacion
    -string bioOrganizacion
    -string sitioWeb
    -bool verificadoOficial
    +obtenerOrganizador(usuarioId) array
    +solicitarVerificacion(usuarioId) bool
}

Usuario <|-- PerfilJugador : "Herencia 1 a 1"
Usuario <|-- PerfilOrganizador : "Herencia 1 a 1"

%% =====
%% ENTIDADES DE COMPETENCIA Y EQUIPOS
%% =====
class Equipo {
    <>
    #int id
    #string nombreEquipo
    #string escudoUrl

```



```

    #int creadoPor
    #string codigoInvitacion
    #bool activo
    +crearEquipo(datos) int
    +transferirCapitania(equipoId, nuevoCapitanId) bool
}

```

```

class EquipoMiembro {
    <>
    #int equipoId
    #int usuarioId
    #DateTime fechaUnion
    #int numeroCamiseta
    #string posicion
    #bool esActivo
    +agregarMiembro(equipoId, usuarioId) bool
    +removeMiembro(equipoId, usuarioId) bool
}

```

```

PerfilJugador "1" --o "N" Equipo : "Capitanea"
Equipo "1" *-- "N" EquipoMiembro : "Tiene Miembros"
PerfilJugador "1" --o "N" EquipoMiembro : "Integra"

```

```

%% =====
%% NÚCLEO DE TORNEOS Y FIXTURE
%% =====

```

```

class Torneo {
    <>
    #int id
    #string nombre
    #int juegoId
    #int organizadorId
    #int modalidadId
    #int sistemaPuntuacionId
    #string formato
    #string estado
    #int cupoMaxEquipos
    +crearTorneo(datos) int
    +cambiarEstado(id, nuevoEstado) bool
    +generarFixture(id) bool
}

```

```

class ParticipanteTorneo {
    <>
    #int id
    #int torneoId
    #string tipo
    #int referenciaId
    #string estado
    +inscribir(torneoId, tipo, refId) bool
    +confirmarCupo(id, confirmadoPor) bool
}

```

```

class Encuentro {

```

```

    <>
    #int id
    #int torneoId
    #int ronda
    #int participanteLocalId
    #int participanteVisitanteId
    #float resultadoLocal
    #float resultadoVisitante
    #string estado
    +registrarMarcador(id, local, visita) bool
    +validarResultado(id, arbitroId) bool
}

class PosicionTorneo {
    <>
    #int torneoId
    #int participanteId
    #int partidosJugados
    #int puntos
    #float diferenciaGoles
    +obtenerTablaOrdenada(torneoId) array
}

PerfilOrganizador "1" --o "N" Torneo : "Organiza"
Torneo "1" *-- "N" ParticipanteTorneo : "Inscribe"
Torneo "1" *-- "N" Encuentro : "Programa"
Torneo "1" *-- "1" PosicionTorneo : "Clasifica"

%% =====
%% MOTORES ALGORÍTMICOS DE TORNEO
%% =====
class ModuloLiga {
    <>
    -float puntosVictoria
    -float puntosEmpate
    +generarFixtureTodosContraTodos(torneoId) bool
    +recalcularTablaPosiciones(torneoId) bool
}

class ModuloEliminacion {
    <>
    -int totalRondas
    +generarArbolLlaves(torneoId) bool
    +avanzarGanador(encuentroId, ganadorId) bool
}

class ModuloSuizo {
    <>
    -string criterioDesempate
    +generarRondaSuiza(torneoId, rondaNumero) bool
    +calcularBuchholz(torneoId, participanteId) float
}

Torneo ..> ModuloLiga : "Usa Formato Liga"

```

```

Torneo ..> ModuloEliminacion : "Usa Formato Eliminacion"
Torneo ..> ModuloSuizo : "Usa Formato Suizo"

%% =====
%% CONTROLADORES Y CAPA MVC
%% =====
class ControladorBase {
    <>
    #renderizar(vista, datos) void
    #responderJson(datos, status) void
}

class AuthControlador {
    <>
    +login() void
    +autenticar() void
    +registro() void
    +logout() void
}

class TorneoControlador {
    <>
    +index() void
    +crear() void
    +generarFixture() void
}

class UsuarioControlador {
    <>
    +index() void
    +crear() void
    +cambiarEstado() void
}

ControladorBase <|-- AuthControlador
ControladorBase <|-- TorneoControlador
ControladorBase <|-- UsuarioControlador

AuthControlador ..> Usuario : "Autentica"
TorneoControlador ..> Torneo : "Gestiona"
UsuarioControlador ..> Usuario : "Gestiona"

%% =====
%% SERVICIOS E INFRAESTRUCTURA
%% =====
class Conexion {
    <>
    -Conexion instancia
    -PDO pdo
    +getInstancia() Conexion
    +getDb() PDO
}

class Sesion {

```

```
<>
+iniciar(usuario) void
+requerirRol(rolNombre) void
+destruir() void
}
```

Usuario ..> Conexion : "Consulta BD"

AuthControlador ..> Sesion : "Inicia Sesion"

4. Instrucciones para Dibujar en Plataformas Visuales

Opción 1: Draw.io

1. Entra a [Draw.io](#).
2. Activa la librería "**UML**" en el panel izquierdo.
3. Arrastra las cajas de "**Class**" (divididas en 3 secciones: Nombre, Atributos, Métodos).
4. Utiliza las flechas conectoras oficiales:
 - **Flecha con triángulo hueco** () para Herencia de **Usuario** a **PerfilJugador**.
 - **Flecha con rombo relleno** () para Composición de **Torneo** a **Encuentro**.
 - **Flecha punteada** () para Dependencia de Controladores a Modelos.

Opción 2: Mermaid Live Editor

1. Abre [Mermaid Live Editor](#).
2. Pega directamente el bloque de código Mermaid de la Sección 3.
3. Exporta la imagen en formato **PNG de alta resolución** o **SVG vectorial**.